



AL MAAREF UNIVERSITY

Mastering Life with Wisdom & Knowledge

Welcome to HackathonTech 2026

Part 2: **Streamlit**





What is Streamlit?

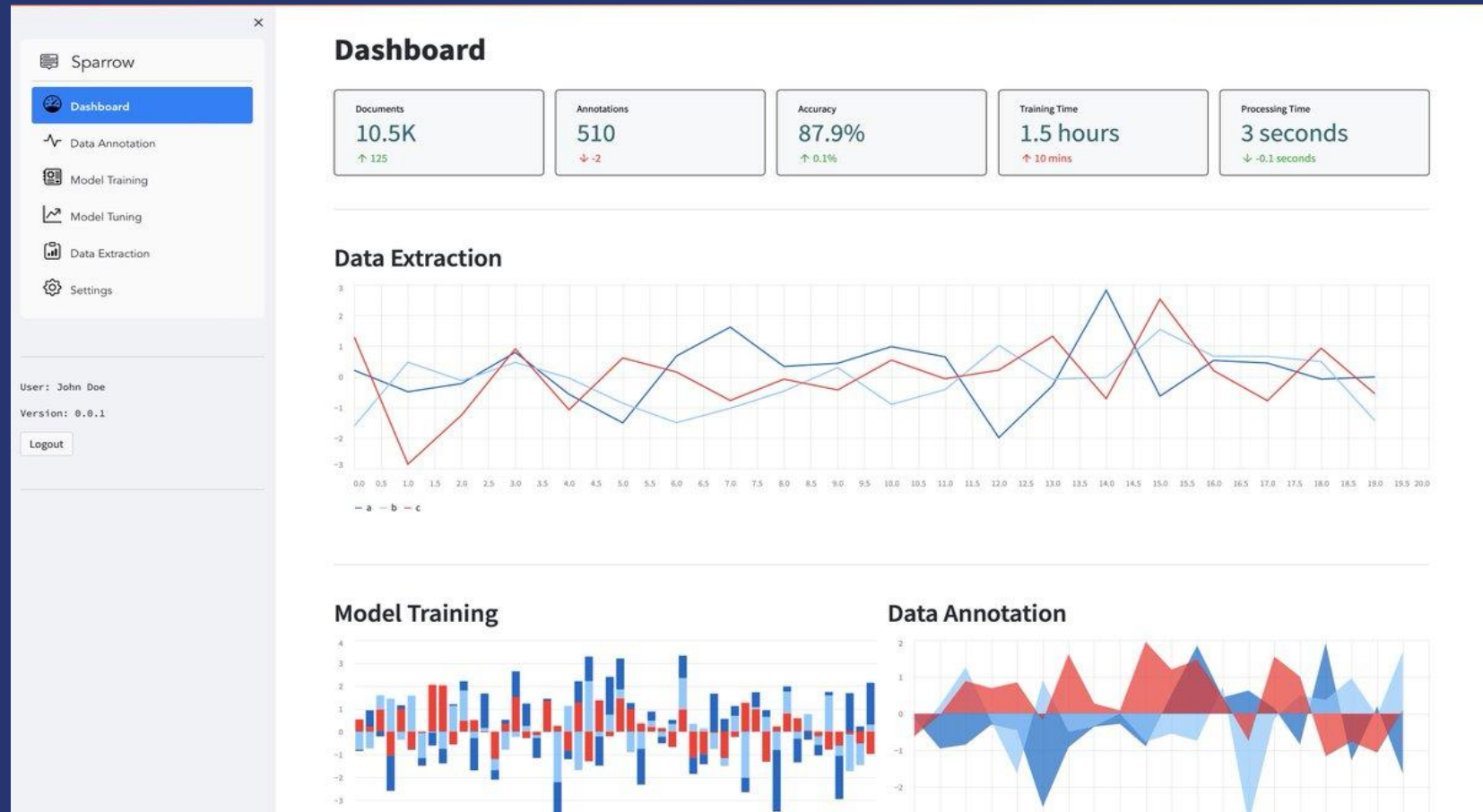
is an open source python library for building interactive web apps using only python

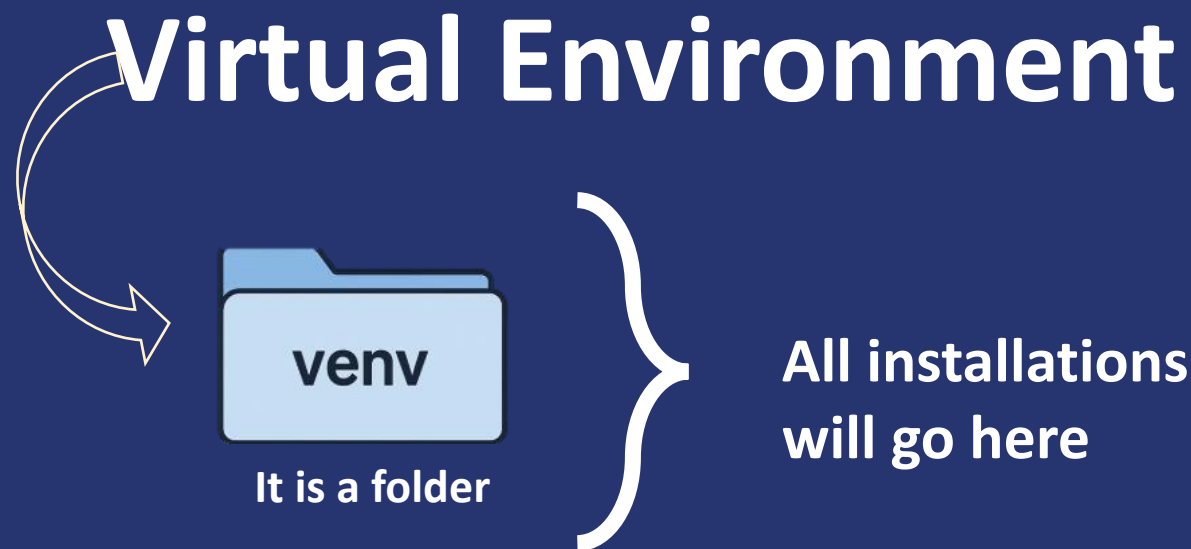
➡ Used mainly to do dashboards and reports





This will be our final outcome







to activate a virtual environment

1 Creating a virtual environment

```
python -m venv .venv - --system-site-packages
```

2 Activating the virtual environment

```
.venv\Scripts\Activate
```

3 Installation Streamlit Package

```
pip install streamlit
```

4 Running Streamlit

```
streamlit run file_name.py
```

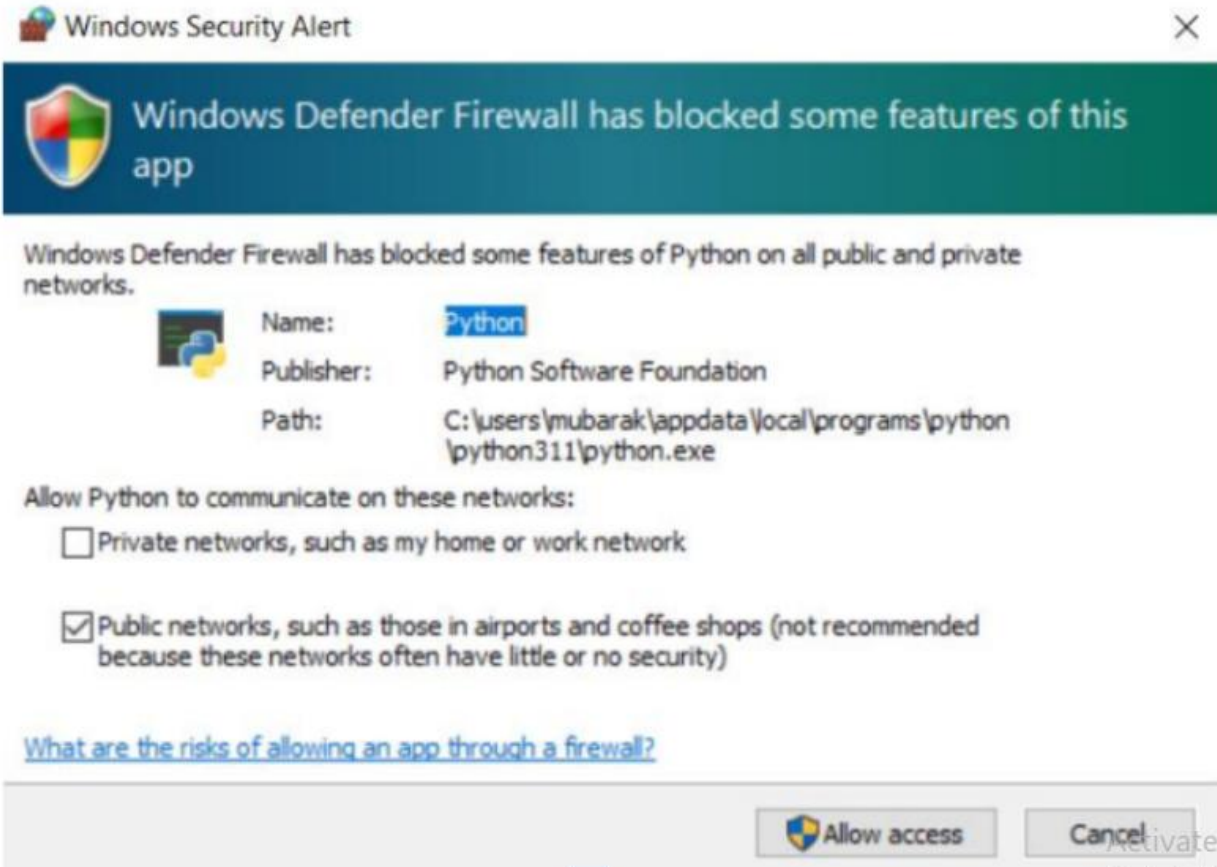


```
(.venv) PS C:\Users\Mubarak\Documents\AI Maaref University\CSC 390 Generative AI\streamlit> streamlit run app1.py

Welcome to Streamlit!

If you'd like to receive helpful onboarding emails, news, offers, promotions,
and the occasional swag, please enter your email address below. Otherwise,
leave this field blank.

Email: email@domain.com
```



Steps while run

Local URL: <http://localhost:8501>

```
(.venv) PS C:\Users\Mubarak\Documents\AI Maaref University\CSC 390 Generative AI\streamlit> streamlit run ap

Welcome to Streamlit!

If you'd like to receive helpful onboarding emails, news, offers, promotions,
and the occasional swag, please enter your email address below. Otherwise,
leave this field blank.

Email: email@domain.com

You can find our privacy policy at https://streamlit.io/privacy-policy

Summary:
- This open source library collects usage statistics.
- We cannot see and do not store information contained inside Streamlit apps,
  such as text, charts, images, etc.
- Telemetry data is stored in servers in the United States.
- If you'd like to opt out, add the following to %userprofile%\streamlit/config.toml,
  creating that file if necessary:

[browser]
gatherUsageStats = false

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.109:8501
```



Understanding Streamlit basic functions

```
import streamlit as st
```

```
st.title("Hello Al Mahdi School")
```

Hello Al Mahdi School !!!



```
st.header("This is a Header- Welcome to HackathonTech")  
st.subheader("This is a subheader - organized by Al Maaref University")
```

Output:

This is a Header - Welcome to HakathonTech

This is a subheader - organized by Al Maaref University



```
st.markdown("# This is a markdown")
st.markdown("## This is a markdown")
st.markdown("### This is a markdown")
st.markdown("#### This is a markdown")
st.markdown("##### This is a markdown")
st.markdown("##### This is a markdown")
```

Output:

This is a markdown

This is a markdown

This is a markdown

This is a markdown

This is a markdown

This is a markdown



```
st.text("Hello all")
st.write("Hello all")
```

Output:

Hello all

Hello all



Success, info, warning & Error



```
st.success("You successfully finished HakathonTeck by A Maaref University")  
st.info("You participated in HakathonTech")  
st.warning("Warning: You will upgrade your skills!!")  
st.error("Error")
```

You successfully finished HakathonTeck by A Maaref University

You participated in HakathonTech

Warning: You will upgrade your skills!!

Error



Input Control



```
name = st.text_input("Enter your name", min_chars=0, placeholder="your name")  
st.write(f"Salam ya, {name}!")
```

Output:

Enter your name

Mahdi

Salam ya, Mahdi!



```
age = st.number_input("Enter your age", min_value=0, max_value=120, value=1)  
st.write(f"you are {age} years old")
```

Enter your age

3

-

+

you are 3 years old



```
temp = st.slider("select temp:", -20,50,20)  
st.write(f"Tempreture: {temp}")
```

select temp:

-20

Tempreture: 20

29

50



```
option= st.selectbox(  
    "choose your favorite color:",  
    ["red", "green", "blue", "yellow"]  
)
```

red

green

blue

yellow

green|





```
agree = st.checkbox("I agree to the terms and conditions")  
if agree:  
    st.success("Thank you for agreeing!")
```

☒ I agree to the terms and conditions

Thank you for agreeing!



```
choice = st.radio(  
    "select your experience level:",  
    ["beginner", "intermediate", "advanced"]  
)  
st.write(f"Experience level: {choice}")
```

select your experience level:

- ☒ beginner
- ☐ intermediate
- ☐ advanced



```
if st.button("Click me"):  
    st.balloons()  
    st.success("Button clicked")
```



```
uploaded_file = st.file_uploader("Choose a file", type=["csv", "txt", "png"])  
if uploaded_file is not None:  
    st.write(f"Uploaded:{uploaded_file.name}")
```



Drag and drop file here

Limit 200MB per file • CSV, TXT, PNG

Browse files



ChatGPT Image Aug 20, 2025, 07_01_22 PM.png 2.7MB



Uploaded:ChatGPT Image Aug 20, 2025, 07_01_22 PM.png



Columns

Purpose arrange content in horizontal columns

```
st.header("Equal with columns")
col1, col2, col3 = st.columns(3)

with col1:
    st.metric("Temp", "70F", "5F")

with col2:
    st.metric("Humidity", "65%", "-3%")

with col3:
    st.metric("Pressure", "1013 hPa", "2 Pa")
```

Output:

Equal with columns

Temp

70F

↑ 5F

Humidity

65%

↓ -3%

Pressure

1013 hPa

↑ 2 Pa



Containers

Purpose : invisible container
to group multiple elements

```
st.header("Grouping Related elements")
data_source_container = st.container()
with data_source_container:
    st.subheader(" data source config")
    source_col1, source_col2 = st.columns(2)
    with source_col1:
        data_type= st.radio("Data Type", ["synthetic", "upload csv", "api"])
    with source_col2 :
        if data_type == "synthetic":
            rows = st.slider("Number of rows", 10,1000,100)
        elif data_type == "upload csv":
            upload_file = st.file_uploader("choose csv")
        else:
            api_url = st.text_input("api endpoint")
```

Grouping Related elements

data source config

Data Type

- ☐ synthetic
- ☒ upload csv
- ☐ api

choose csv

Drag and drop file here

Limit 200MB per file

Browse files

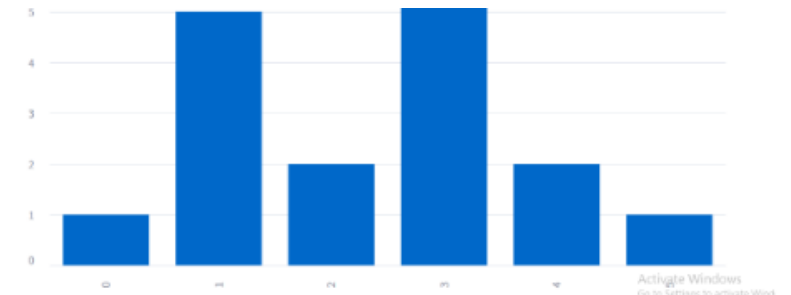
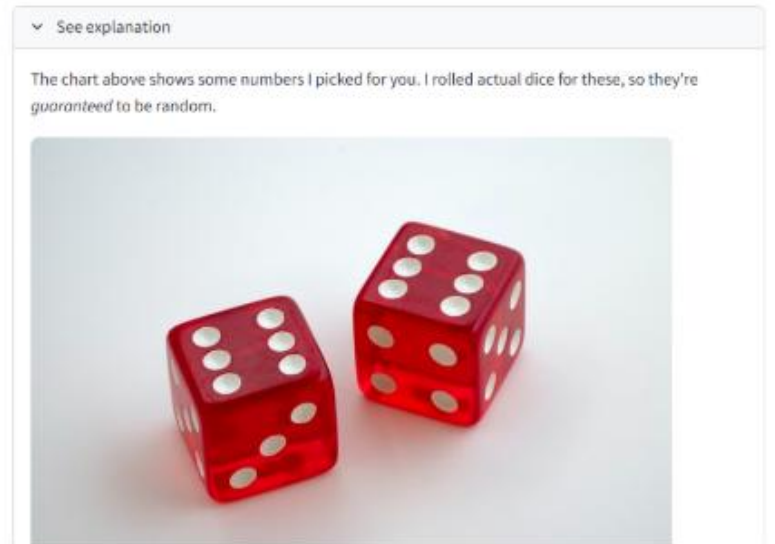
Expander

Purpose : create toggleable content sections



```
st.bar_chart({"data": [1, 5, 2, 6, 2, 1]})

with st.expander("See explanation"):
    st.write(''
        The chart above shows some numbers I picked for you.
        I rolled actual dice for these, so they're *guaranteed* to
        be random.
    '')
    st.image("https://static.streamlit.io/examples/dice.jpg")
```





Navigation

```
page = st.navigation(  
    [  
        st.Page("pages/home.py", title="Home"),  
        st.Page("pages/about.py", title="About"),  
        st.Page("pages/courses.py", title="Courses"),  
        st.Page("pages/contacts.py", title="Contact Us")  
    ],  
    position="top"  
)  
page.run()
```

pages

- about.py
- contacts.py
- courses.py
- home.py



Session State

Streamlit apps are **declarative**

Every time a user interacts with a widget, Streamlit reruns the entire script from top to bottom

Variables reset each run

You can not remember what was entered last time unless you store it somewhere persistent

Session
state

It acts like a Python Dictionary that persists across reruns for a single user session



Session State

```
import streamlit as st

st.title("Understanding Session state")
if "counter" not in st.session_state:
    st.session_state.counter=0
count = 0
if st.button("Increment"):
    st.session_state.counter+=1
    count+=1
if st.button("Reset"):
    st.session_state.counter=0
    count+=0

st.markdown(f'### Session state update value {st.session_state.counter}')
st.markdown(f'### Count value {count}')
```

Understanding Session state

Increment

Reset

Session state update value 10

Count value 1

Each rerun preserve the `st.session_state.counter` variable. It behaves like a global variable



Session State



```
name = st.text_input("Enter your name", key="username")  
st.write("Hello", st.session_state.username)
```

Why session_state? Not a normal variable?

To remember the data across the app



Callbacks

Sometimes you do not want to wait for a button press, you want code to automatically execute when a value changes

Streamlit widgets support
callback functions

- 1 on_change -> for input widget
- 2 on_click -> for buttons
- 3 on_click -> for forms button submission

Each callback points to a function that runs immediately when
that event occurs



Callbacks on_change

```
import streamlit as st

def quadrable():
    st.session_state.quad = st.session_state.value**4

st.number_input("Enter you number",
                key="value",
                on_change=quadrable,
                min_value=1,
                max_value=30)

st.success(st.session_state.get("quad", "waiting for input"))
```

Key="value" => sets the session state variable
on_change=quadrable=> the callback function

Enter you number

5

-

+

625



Callbacks on_click

```
import streamlit as st

def add_to_history():
    msg=st.session_state.new_msg

    if "history" not in st.session_state:
        st.session_state.history=[]

    st.session_state.history.append(msg)
    st.session_state.new_msg = ""

st.text_input("Enter message", key="new_msg")
st.button("Add", on_click=add_to_history)
st.write("Chat History", st.session_state.get("history", []))
```

Enter message

Add

Chat History

```
[
  0 : "hi"
  1 : "how are you"
]
```




Callbacks on_click

```
import streamlit as st

st.set_page_config(page_title="Chat Example", layout="wide")
st.title("Sample chatbot with session state and callbacks")

if "messages" not in st.session_state:
    st.session_state.messages=[]

def add_message():
    msg= st.session_state.input_msg
    if msg.strip() != '':
        st.session_state.messages.append({"role":"user", "content":msg})
        st.session_state.messages.append({"role":"assistant", "content":f"Echo {msg}"})
        st.session_state.input_msg = ''

st.text_input("Type a message: ", key="input_msg", on_change=add_message)

for m in st.session_state.messages:
    if m["role"] == "user":
        st.write(f"**You** {m['content']}")
    else:
        st.write(f"**AI** {m['content']}")
```



Callbacks on_click

Sample chatbot with session state and callbacks [↗](#)

Type a message:

You hi

AI Echo hi

You how are you

AI Echo how are you



Forms

If a widget is not inside a form
-> the program runs again automatically
whenever the user changes its value

If a widget is inside a form
-> the app does reruns when the user clicks
the submit button

Forms

Inside the form

Enter your name:

kawthar

Pick a number



Pick a color

red

Submit my picks

```
{'name': 'kawthar', 'number': 6, 'color': 'red'}
```

```
import streamlit as st

if "choice" not in st.session_state:
    st.session_state.choice = {}

def process_data():
    values = {}
    values["name"] = st.session_state.name
    values["number"] = st.session_state.number
    values["color"] = st.session_state.color

    st.session_state.choice = values

with st.form("data_form"):
    st.write("Inside the form")

    st.text_input("Enter your name:", key="name")
    st.slider("Pick a number", 1, 10, key="number")
    st.selectbox(
        "Pick a color",
        ["red", "orange", "green", "blue", "violet"],
        key="color"
    )

    st.form_submit_button("Submit my picks", on_click=process_data)

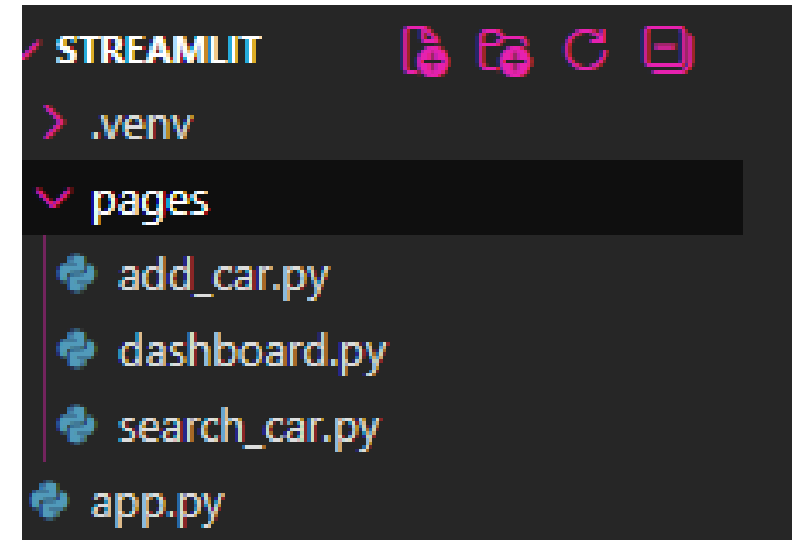
if st.session_state.choice:
    st.success(st.session_state.choice)
else:
    st.info("Waiting for data ...")
```



Multi page Streamlit Application

For a car management system

1. Create a page with a Form to pick essential data input from the user using different types of widgets and store it in session state.
2. Create a page to search for data from your saved state and show details if available
3. Create a dashboard and show some statistics based on your data using a graph
4. Use different layouts: collapse, columns, and containers



app.py



```
import streamlit as st

if "cars" not in st.session_state:
    st.session_state.cars = []

page = st.navigation(
    [
        st.Page("pages/add_car.py", title="Add Car"),
        st.Page("pages/search_car.py", title="Search Car"),
        st.Page("pages/dashboard.py", title="Dashboard")
    ],
    position="top"
)
page.run()
```

add_car.py

```
import streamlit as st

st.title("Add a New Car")

with st.form("add_car_form"):
    col1, col2 = st.columns(2)

    with col1:
        car_make = st.text_input("Car Make (e.g., Toyota)")
        car_model = st.text_input("Car Model (e.g., Corolla)")

    with col2:
        car_year = st.number_input("Year", min_value=1990, max_value=2030, step=1)
        car_color = st.selectbox(
            "Color", ["Red", "Blue", "Black", "White", "Silver", "Other"]
        )

    car_price = st.slider("Price ($)", min_value=5000, max_value=100000, step=500)
    car_available = st.checkbox("Available for Sale?", value=True)
    submitted = st.form_submit_button("Add Car")

    if submitted:
        st.session_state.cars.append({
            "Make": car_make,
            "Model": car_model,
            "Year": car_year,
            "Color": car_color,
            "Price": car_price,
            "Available": car_available
        })
    st.success(f"Car {car_make} {car_model} added successfully!")
```

search_car.py

```
import streamlit as st
import pandas as pd

st.title("Search Cars")

if "cars" not in st.session_state:
    st.session_state.cars = []

search_make = st.text_input("Search by Make")
search_model = st.text_input("Search by Model")

if st.button("Search"):
    results = [
        car for car in st.session_state.cars
        if (search_make.lower() in car["Make"].lower()) and
           (search_model.lower() in car["Model"].lower())
    ]
    if results:
        st.write("### Search Results:")
        st.dataframe(pd.DataFrame(results))
    else:
        st.warning("No cars found matching your search.")
```


Dashboard.py

```
import streamlit as st
import pandas as pd

st.title("Cars Dashboard")
df = pd.DataFrame(st.session_state.cars)

if not df.empty:
    col1, col2, col3 = st.columns(3)
    col1.metric("Total Cars", len(df))
    col2.metric("Available Cars", df['Available'].sum())
    col3.metric("Average Price", f"${int(df['Price'].mean())}")

    st.subheader("Average Price by Car Make")
    st.bar_chart(df.groupby("Make")["Price"].mean())

    st.subheader("Cars by Color")
    st.bar_chart(df['Color'].value_counts())
else:
    st.info("No cars added yet. Add some cars to see the dashboard.")
```



Congratulations!

New Milestone 2/4